

Object Oriented Programming (CS1004)

Date: April 6 2026

Course Instructor(s)

SK, UA, UN, AK, HI

Sessional-II Exam

Total Time (Hrs): 1

Total Marks: 30

Total Questions: 2

Roll No

Section

Student Signature

Do not write below this line

Attempt all the questions on question paper. You can use rough sheet but do not attach it with the question paper. Don not use red ink or pencil to solve the exam.

CLO #3: Model an Algorithmic Solution for a given problem using OOP

Q1: You are required to design and implement a C++ program that models a simple **library catalog system**. The program must consist of two classes: **Book** and **BookCatalog**. Each book has a title, ISBN and author name. The BookCatalog class must internally use a **pointer to a dynamically allocated array of Book objects**.

Implement the program so that the below main() runs correctly. Make sure to apply all OOP principals and use dynamic memory responsibly (no memory leaks no dangling pointers). You can use string data type. **[20 Marks]**

```
Int main() {
    BookCatalog catalog1, catalog2; //must create empty catalog

    //First asks the number of books in the catalog and then input desired number of books
    cin >> catalog1;
    cin >> catalog2;

    // Output catalogs (all the books in the catalogue)
    cout << catalog1 << catalog2;

    cout << "First book in catalog1: " << catalog1[0] << endl;
    //checks whether two catalogs have same books in same order or not
    if (catalog1 == catalog2)
        cout << "Catalogs are equal.\n";
    else
        cout << "Catalogs are not equal.\n";

    // Merge catalog must include all the books of catalog1 and catalog2
    BookCatalog merged = catalog1 + catalog2;
    cout << "Merged catalog:\n" << merged;

    Book newBook("The Pragmatic Programmer",1000, "Andrew Hunt");
```

National University of Computer and Emerging Sciences

Lahore Campus

```
// Add a single book to the catalog1 and store it in another object of catalog
BookCatalog extended = newBook + catalog1;
cout << "Extended catalog:\n" << extended;

BookCatalog copyCatalog;
copyCatalog = catalog1;
cout << "Copy of Catalog 1:\n" << copyCatalog;

BookCatalog tempCatalog = catalog2 + Book("TempBook", 2000,"TempAuthor");
BookCatalog movedCatalog;
movedCatalog = std::move(tempCatalog);
cout << "Moved Catalog:\n" << movedCatalog;

// Inequality operator
if (copyCatalog != merged)
    cout << "CopyCatalog and MergedCatalog are different.\n";
return 0;
}
```

```
class Book {
    string title;
    int ISBN;
    string author;
public:
    Book(string t, int i, string a) : title(t), ISBN(i),
    author(a) {}
    friend istream& operator>>(istream& in,
    Book& b) {
        cout << "Enter title, ISBN and author\n ";
        in >> b.title;
        in >> b.ISBN;
        in >> b.author;
        return in;
    }
    friend ostream& operator<<(ostream& out,
    const Book& b) {
        out << "Title: " << b.title << ", ISBN: " <<
        b.ISBN << ", Author: " << b.author;
        return out;
    }
    bool operator==(const Book& other) const {
        return (title == other.title && ISBN ==
        other.ISBN && author == other.author);
    }
    bool operator!=(const Book& other) const {
        return !(*this == other);
    }
};
```

```
class BookCatalog {
    Book* books;
    int size;
public:
    BookCatalog() : books(nullptr), size(0) {}
    // Copy constructor
    BookCatalog(const BookCatalog& other) {
        size = other.size;
        books = new Book[size];
        for (int i = 0; i < size; i++) {
            books[i] = other.books[i];
        }
    }
    // Move constructor
    BookCatalog(BookCatalog&& other) {
        size = other.size;
        books = other.books;
        other.books = nullptr;
        other.size = 0;
    }
    // Destructor
    ~BookCatalog() {
        delete[] books;
    }
    bool operator==(const BookCatalog& other)
    const {
        if (size != other.size) return false;
        for (int i = 0; i < size; i++) {
```

National University of Computer and Emerging Sciences

Lahore Campus

<pre> // Copy assignment BookCatalog& operator=(const BookCatalog& other) { if (this != &other) { delete[] books; size = other.size; books = new Book[size]; for (int i = 0; i < size; i++) { books[i] = other.books[i]; } } return *this; } // Move assignment BookCatalog& operator=(BookCatalog&& other) noexcept { if (this != &other) { delete[] books; size = other.size; books = other.books; other.books = nullptr; other.size = 0; } return *this; } friend istream& operator>>(istream& in, BookCatalog& catalog) { cout << "Enter number of books: "; in >> catalog.size; delete[] catalog.books; catalog.books = new Book[catalog.size]; for (int i = 0; i < catalog.size; i++) { cout << "Book " << i + 1 << ":\n"; in >> catalog.books[i]; } return in; } friend ostream& operator<<(ostream& out, const BookCatalog& catalog) { for (int i = 0; i < catalog.size; i++) { out << catalog.books[i] << endl; } return out; } Book& operator[](int index) { return books[index]; } </pre>	<pre> if (books[i] != other.books[i]) return false; } return true; } bool operator!=(const BookCatalog& other) const { return !(*this == other); } // Merge two catalogs BookCatalog operator+(const BookCatalog& other) const { BookCatalog result; result.size = size + other.size; result.books = new Book[result.size]; for (int i = 0; i < size; i++) result.books[i] = books[i]; for (int j = 0; j < other.size; j++) result.books[size + j] = other.books[j]; return result; } // Add a single book to catalog friend BookCatalog operator+(const Book& b, const BookCatalog& catalog) { BookCatalog result; result.size = catalog.size + 1; result.books = new Book[result.size]; result.books[0] = b; for (int i = 0; i < catalog.size; i++) { result.books[i + 1] = catalog.books[i]; } return result; } friend BookCatalog operator+(const BookCatalog& catalog, const Book& b) { BookCatalog result; result.size = catalog.size + 1; result.books = new Book[result.size]; for (int i = 0; i < catalog.size; i++) { result.books[i] = catalog.books[i]; } result.books[catalog.size] = b; return result; } }; </pre>
---	--

National University of Computer and Emerging Sciences

Lahore Campus

CLO #1: Demonstrate basic concepts of OOP

Q2: Answer the following:

[5+5 Marks]

<pre> class Test{ int x; int y; public: Test(int x = 0, int y = 0) { this->x = x; this->y = y; } Test& setX(int a) { x = a; return *this; } Test* setY(int b) { y = b; return this; } void print() { cout << "x = " << x << " y = " << y << endl; } }; </pre>	<pre> class Demo { const int id; int a; static int b; public: Demo(int i, int j) { id = i; a = j; b++; } static void show() { cout << a << b << endl; } }; int Demo::b = 0; int main() { Demo d(5,2); d.show(); return 0; } </pre>
Which of the following statement(s) is valid and what would be the output; <code>(**)(obj1.setX(10).setY(20)).setX(23).setY(12)).print();</code> <code>obj1.setX(10).setY(20)->setX(23).setY(12).print();</code> <code>(*)(*obj1.setX(10).setY(20)).setX(23).setY(12)).print();</code> <code>(**obj1.setX(10).setY(20).setX(23).setY(12)).print();</code> III is correct out put is x=23 y=12	Identify the error in the following code and suggest correction. Const data member id not initialized properly. Use member initializer list Static member function cannot access non-static member. Make show non static